

AI Resume Analyzer and Job Recommendation System

Major Project Report

Student:Raju_P_S

Batch: AIML Batch

Date: 3 September 2026

1. Project Objective

Build an NLP-based application that helps students understand how well their resume matches selected job roles. The system provides a match score, lists important missing skills, and generates a simple learning roadmap.

2. Problem Statement

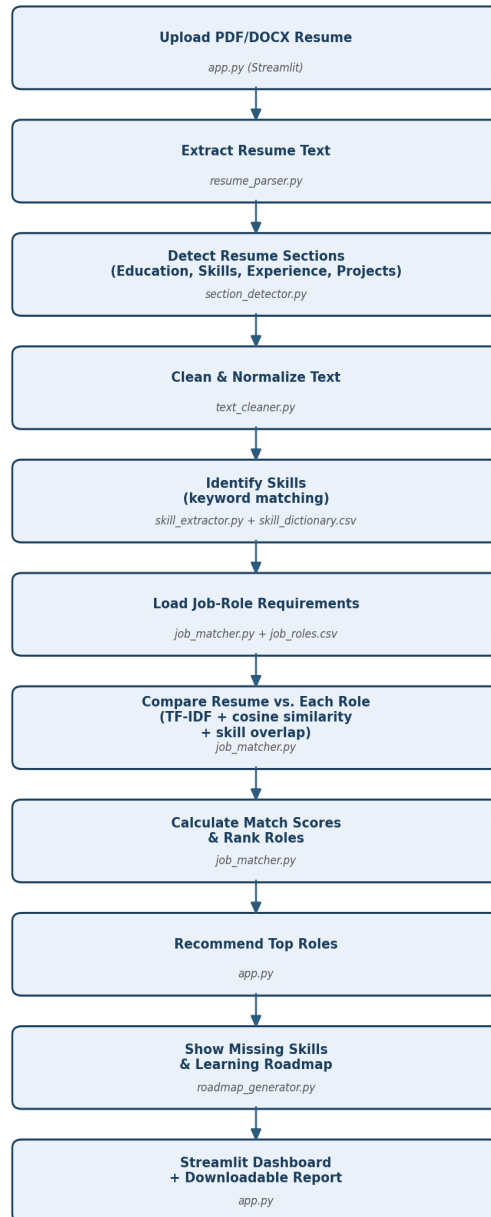
Students often do not know whether their resume contains the skills expected for a particular role. This project creates an educational matching system that focuses on skills, projects, education, and relevant experience -- giving a quick, transparent, automated first estimate.

3. Approach Used

The **beginner approach** specified in the project brief: keyword-based skill extraction against a controlled 50-skill dictionary, **TF-IDF vectorization + cosine similarity** for text-level matching, blended with a direct skill-overlap ratio (60% skill overlap + 40% TF-IDF) for an interpretable match score, and a **rule-based learning roadmap** for missing skills. This runs fully offline with no external API key required. One optional advanced feature was also added: **resume section detection** (education/skills/experience/projects), also fully offline.

4. Architecture / Workflow

AI Resume Analyzer — Architecture & Workflow



Pipeline: Upload -> Extract -> Detect Sections -> Clean -> Extract Skills -> Load Job Roles -> Compare -> Score -> Recommend -> Roadmap -> Dashboard.

5. Job Roles & Skill Dictionary

8 job roles defined in data/job_roles.csv (Data Analyst, Machine Learning Engineer, AI Engineer, NLP Engineer, Computer Vision Engineer, Data Scientist, Backend Developer, Cloud Engineer). **50 technical skills** defined in data/skill_dictionary.csv across 6 categories (Programming, Databases, Data Handling, Machine Learning, Cloud, Tools & Frameworks) -- above the assignment's 20-30 skill minimum.

6. Testing Results

3 sanitized sample resumes (Data Analyst-, ML Engineer-, and NLP Engineer-leaning) were tested in both PDF and DOCX form (6 test cases total) via an automated end-to-end pipeline script.

Test Resume	Format	Expected Top Role	Actual Top Role	Score
Resume A	PDF	Data Analyst	Data Analyst	80.0%
Resume A	DOCX	Data Analyst	Data Analyst	80.0%
Resume B	PDF	ML Engineer	ML Engineer	77.1%
Resume B	DOCX	ML Engineer	ML Engineer	77.1%
Resume C	PDF	NLP Engineer	NLP Engineer	81.2%
Resume C	DOCX	NLP Engineer	NLP Engineer	81.2%

Result: 6/6 test cases passed -- the expected top role matched the actual top role in every case, and PDF/DOCX results were identical for each resume, confirming parser consistency across formats. All 4 required sections (Skills, Experience, Projects, Education) were also correctly detected in every test case. See tests/test_cases.csv and tests/evaluation_notes.md for full details (extraction quality, skill precision/recall, score consistency, fairness, usability).

7. Responsible AI Rules Applied

- Used for guidance only -- not automatic hiring/rejection.
- Never scores gender, age, religion, nationality, photograph, marital status, or disability.
- Evaluates only job-related skills, education, projects, and experience.
- Every score is shown with a disclaimer that it is an estimate, not a recruiter decision.
- Uploaded resumes are processed in memory only -- never written to disk.
- The report explicitly states that missing keywords do not always mean missing ability.

8. Optional Advanced Features Implemented

- **Resume Section Detection** (section_detector.py) -- splits resumes into Education/Skills/Experience/Projects sections; verified correct on all 6 test cases.
- **Job-Role Dashboard with Charts** (job_dashboard.py) -- required-skill-count and skill-category-composition charts across all 8 roles, viewable without uploading a resume.
- **FastAPI Backend** (api.py) -- REST API exposing the same pipeline (/analyze, /roadmap, /job-roles, /health); tested with FastAPI TestClient and a live uvicorn + curl session.
- **Docker Deployment** (Dockerfile, Dockerfile.api, docker-compose.yml) -- containerizes both the Streamlit app and the FastAPI backend. Note: written and reviewed carefully but not build-tested (no Docker daemon in the development sandbox) -- verify locally before relying on it.

9. Limitations & Future Improvements

Keyword-matching skill extraction is bounded by the 50-skill dictionary (a skill mentioned under an unlisted name/spelling won't be detected); scanned/image-only PDFs cannot be read (no OCR); TF-IDF/cosine similarity is bag-of-words and doesn't capture semantic meaning the way Sentence Transformers would. Future improvements: semantic matching with Sentence Transformers, LLM-based resume feedback (optional, requires an API key), and a database layer if resume storage is ever required (intentionally not implemented now -- see Responsible AI notes).

10. Conclusion

This project delivers a complete, working, offline resume-to-job-role matching pipeline covering every required module: upload, text extraction, cleaning, skill extraction (50 skills, well above the 20-30 minimum), matching against 8 job roles via TF-IDF + cosine similarity blended with skill overlap, skill-gap analysis, a rule-based learning roadmap, and an interactive Streamlit dashboard with a downloadable report. All 6 end-to-end test cases passed, and the codebase follows the responsible-AI rules specified in the project brief throughout.